Trabalho 1: Busca Adversarial em Jogos
Agente Inteligente para Breakthrough

Pedro Henrique dos Reis Arcoverde - 254719
Pedro Henrique Pinheiro Linhares - 175807 Nome do Aluno 3
Nome do Aluno 4 Nome do Aluno 5

Março de 2025

1 Modelagem do Problema

1.1 Apresentação do Problema: Breakthrough

O Breakthrough é um jogo de tabuleiro estratégico de soma zero e informação perfeita. O jogo ocorre em um tabuleiro quadriculado, onde cada jogador começa com duas fileiras de peças. As regras de movimentação ditam que as peças movem-se apenas para frente, ou na diagonal para capturar peças inimigas. O jogo possui duas condições de vitória: vence quem alcançar a última fileira adversária ou eliminar todas as peças do oponente.

1.2 Definição Formal do Jogo

Para modelar o Breakthrough como um problema de busca em Inteligência Artificial, definimos formalmente o jogo como a tupla (S, A, T, U):

- **Estados (S):** O conjunto de todas as configurações possíveis do tabuleiro. Cada estado é representado por uma matriz 8×8, onde cada célula pode conter: vazio (0), peça do Jogador 1 (1) ou peça do Jogador 2 (2).
- **Ações (A):** As jogadas legais disponíveis para um jogador em um dado estado. Uma ação é representada pela coordenada de origem da peça e o seu destino.
- **Modelo de Transição (T):** A função $T(s, a) = s'$, que recebe um estado s e uma ação legal a, e retorna um novo estado s'.
- **Função de Utilidade (U):** A função que avalia os estados. Retorna um valor numérico máximo se o estado resulta em vitória para o agente, um valor mínimo se resulta em derrota, e avaliações heurísticas intermediárias para estados não terminais.

1.3 Teste de Terminal

O teste de estado terminal verifica as condições de fim de jogo. A busca é interrompida naquele ramo se houver invasão (uma peça alcançou a fileira base do inimigo) ou aniquilação (um jogador perdeu todas as peças).

2 Algoritmos de Busca Utilizados

Para dotar o agente da capacidade de tomar decisões competitivas, foram implementados algoritmos clássicos de busca adversarial, respeitando o limite orçamentário de tempo por jogada e utilizando *Iterative Deepening*.

2.1 Minimax Puro

O algoritmo Minimax assume que ambos os jogadores jogam de forma otimizada. O agente (Max) busca maximizar sua função de utilidade, enquanto o oponente (Min) busca minimizá-la. Seu custo computacional é $\mathcal{O}(b^d)$, onde b é o fator de ramificação e d é a profundidade. Sem podas, o Minimax puro se comporta fazendo uma busca exaustiva em cada ramificação da árvore do jogo o que acaba sendo inviável para profundidades maiores devido à explosão combinatória, como também, não é funcional dados os limites de tempo impostos por jogada.

```
Função MINIMAX(estado, profundidade, maximizando):
    Se profundidade == 0 ou TESTE-TERMINAL(estado):
        Retorna AVALIAÇÃO-HEURÍSTICA(estado)

    Se maximizando:
        max_eval = -INFINITO
        Para cada ação em GERAR-AÇÕES(estado):
            novo_estado = TRANSICAO(estado, ação)
            eval = MINIMAX(novo_estado, profundidade - 1, Falso)
            max_eval = MAX(max_eval, eval)
        Retorna max_eval
    Senão:
        min_eval = +INFINITO
        Para cada ação em GERAR-AÇÕES(estado):
            novo_estado = TRANSICAO(estado, ação)
            eval = MINIMAX(novo_estado, profundidade - 1, Verdadeiro)
            min_eval = MIN(min_eval, eval)
        Retorna min_eval
```

2.2 Alpha-Beta Pruning (Poda Alfa-Beta)

A Poda Alfa-Beta é uma otimização obrigatória do Minimax que evita explorar ramos da árvore que não influenciarão a decisão final. Utiliza os parâmetros α (melhor escolha garantida para Max) e β (melhor escolha garantida para Min). Se em algum momento o algoritmo descobre que um estado pioraria a garantia já estabelecida ($\beta \leq \alpha$), o ramo inteiro é podado, ou seja, eliminado.

Isso reduz a complexidade para até $\mathcal{O}(b^{d/2})$ no melhor caso. Operando sob o mesmo limite fixo por jogada, o agente consegue atingir maiores profundidades.

```
Função ALPHA-BETA(estado, profundidade, alfa, beta, maximizando):
    Se profundidade == 0 ou TESTE-TERMINAL(estado):
        Retorna AVALIAÇÃO-HEURÍSTICA(estado)
```

```
    Se maximizando:
        max_eval = -INFINITO
        Para cada ação em GERAR-AÇÕES(estado):
            novo_estado = TRANSICAO(estado, ação)
            eval = ALPHA-BETA(novo_estado, profundidade - 1, alfa, beta, Falso)
            max_eval = MAX(max_eval, eval)
            alfa = MAX(alfa, eval)
            Se beta <= alfa:
                Interrompe o laço (PODA)
        Retorna max_eval
    ... (lógica espelhada para MIN)
```

2.3 Estratégia de Ordenação de Jogadas (Move Ordering)

Para que a poda atinja seu desempenho máximo, implementou-se a ordenação de jogadas obrigatória. Lances que resultam em captura de peças adversárias recebem prioridade. Ao avaliar capturas primeiro, é muito provável que o algoritmo encontre rapidamente um valor elevado de α, aumentando massivamente o número de podas prematuras (*cut-offs*).

```
Função ORDENAR-JOGADAS(estado, acoes):
    Para cada ação em acoes:
        ação.score = 0
        Se TIPO-MOVIMENTO(ação) == CAPTURA:
            ação.score = 10
    Retorna acoes ordenadas de forma decrescente pelo score
```

3 Funções Heurísticas

Em qualquer tomada de decisão, alguns aspectos são levados em conta. Até para dar um passo, inconscientemente, são calculadas as possibilidades que essa decisão pode gerar. Desta forma, é possível criar um algoritmo que calcule o quão atrativa uma decisão pode ser. Para o caso do Breakthrough, é essencial escolher uma heurística, pois o número de combinações pode alcançar valores exponenciais. Portanto, decidimos utilizar duas heurísticas diferentes para poder comparar quais características são melhores e, assim, aumentar as chances de vitória no jogo.

3.1 Diferença de Material

Esta heurística foca em calcular a diferença do número de peças entre os oponentes. Portanto, quanto mais peças um jogador tiver, mais chance ele terá de vencer, desta forma, o algoritmo foca em escolher o estado que ele possui mais chance de vencer.

```
Função Diferença_de_Material(estado, jogador):
    score = 0
    Para posição em estado:
        Se valor_posição == 1:
            score += 1
        Se valor_posição == 2:
```

```
        score -= 1
    Se jogador == 2:
        score = score * (-1)
    Retorna score
```

3.2 Avanço Médio das Peças

Esta heurística foca em calcular o quão vantajoso uma jogada é a partir da distância das peças até o final do tabuleiro. Desta forma, ao longo do jogo cada peça possui uma influência maior na vitória, por exemplo, uma peça que está mais próxima da última fileira do oponente acrescenta um valor maior ao peso desta peça. Por fim, precisamos apenas somar o peso de todas as peças de um jogador e sobtrair pela soma do peso das peças do outro jogador.

```
Função Avanço_Médio_das_Peças(estado, jogador):
    score = 0
    total_peças = 0
    total_por_linha = 0
    Para posição em estado:
        Se jogador == 1:
            total_peças += 1
            total_por_linha += 1 + ((posição + 1) // 8))
        Se jogador == 2:
            total_peças += 1
            total_por_linha += 1 + (7 - (posição + 1) // 8))
    Retorna score
```